Vigilante de vulnerabilidades para webs en Docker

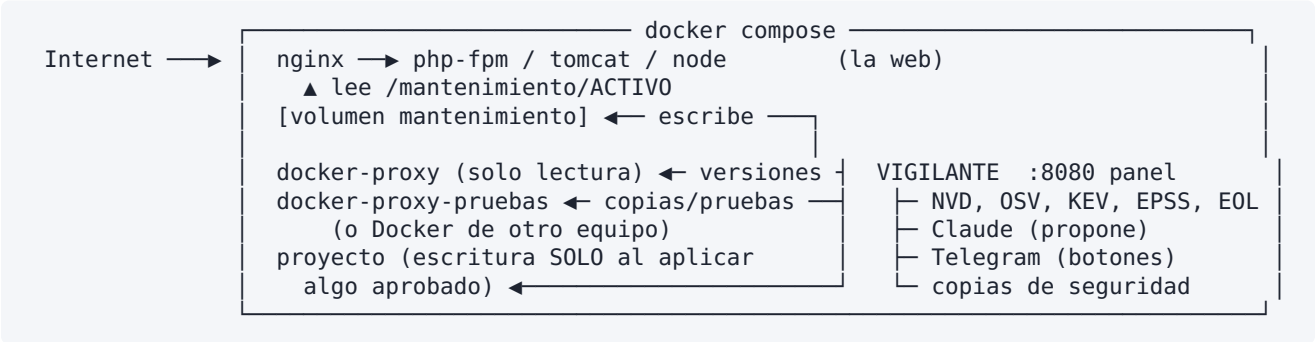
Diseño y prototipo · v2: corrección con Claude con aprobación del usuario

27 de octubre de 2026

1. Qué es

Un contenedor que corre **siempre al lado de una web ya desplegada**. Al arrancar (y luego cada pocas horas):

1. **Detecta** qué versiones exactas hay desplegadas: PHP, nginx, Java, Tomcat, Node…, y las dependencias del proyecto (Composer, npm, Maven…).
2. **Consulta** fuentes públicas de vulnerabilidades buscando las que afectan a esas versiones.
3. **Prioriza**: no es lo mismo un CVE teórico que uno que se está explotando ahora mismo.
4. **Avisa siempre**, por consola y por los canales que se configuren (Telegram, webhook, email).
5. Si hay clave de API de Anthropic, el usuario puede **pedirle a Claude** (nunca lo hace solo) que proponga los cambios que lo corrigen, cada uno justificado. El vigilante los **verifica en una copia** con contenedores de prueba. **Nada se aplica sin aprobación explícita** del usuario (Telegram o panel). Una vez aprobado, se aplica con copia de seguridad previa y posibilidad de volver atrás (§6).
6. Si la vulnerabilidad es crítica, permite **poner la web en modo mantenimiento**, con un botón o de forma automática.



2. Cómo detectar las versiones desplegadas

La clave es no fiarse de una sola pista. El vigilante combina **cuatro capas**, de más a menos fiable, y cuando dos capas hablan del mismo componente gana la más fiable.

Capa	Qué mira	Ejemplo	Fiabilidad	Invasividad
A. Sondas dentro del contenedor (opcional)	`php -r 'echo PHP_VERSION;'`, `nginx -v`, `java -version`… vía `docker exec`	`nginx/1.25.3`	Muy alta: es lo que corre de verdad	Media (necesita permiso de exec)

Capa	Qué mira	Ejemplo	Fiabilidad	Invasividad
B. Variables de entorno de la imagen	Las imágenes oficiales definen `PHP_VERSION`, `NGINX_VERSION`, `JAVA_VERSION`, `TOMCAT_VERSION`, `NODE_VERSION`, `PG_VERSION`…	`PHP_VERSION=8.1.2`	Alta	Nula (solo inspect)
C. Etiqueta de imagen y Dockerfile	`php:8.2.10-fpm`, líneas `FROM`	`FROM php:8.1.2-fpm`	Media: `latest` o `8.2` no dicen la versión exacta	Nula
D. Ficheros «lock» del proyecto	`composer.lock`, `package-lock.json`, `yarn.lock`, `pom.xml`, `requirements.txt`, `Gemfile.lock`, `go.mod`, `wp-includes/version.php`	`guzzlehttp/guzzle 7.4.0`	Alta para dependencias: el lock es la versión instalada	Nula (montaje de solo lectura)
E. Desde fuera (HTTP)	Cabeceras `Server`, `X-Powered-By`, `<meta generator>`	`Server: nginx/1.18.0`	Baja: muchas webs lo ocultan (bien hecho)	Nula
Manual	`[deteccion.manual]` en la configuración	`tomcat = "9.0.50"`	La que diga el usuario	—

Qué contenedores se vigilan. Por orden: los que se pongan por nombre en la configuración; si no, los que tengan la etiqueta `vigilante.vigilar=true`; y si no hay ninguno, el resto de contenedores del mismo proyecto `docker compose`.

Por qué a través de un proxy del socket de Docker. Montar `/var/run/docker.sock` equivale a dar acceso de root sobre el servidor. El ejemplo usa `tecnativa/docker-socket-proxy`, que solo deja listar e inspeccionar contenedores (las capas B y C). La capa A (exec) va desactivada por defecto y se activa a propósito.

Versiones imprecisas. Si solo se sabe `php 8.1` (p. ej. de un `FROM php:8.1-fpm`), no se buscan CVE por versión exacta, porque daría falsos positivos y negativos. Sí se comprueba el fin de soporte de la rama.

3. De dónde sacar los CVE

No hay una fuente que lo cubra todo bien. La combinación elegida, gratuita y sin registro, es:

Fuente	Para qué	Cómo se consulta	Notas
NVD (NIST)	CVE de runtimes y servidores: PHP, nginx, Apache, Tomcat, OpenJDK, Node, MySQL, PostgreSQL, Redis, WordPress…	API 2.0 `virtualMatchString=cpe:2.3:a:php:php:8.1.2`	Es la referencia oficial de CVE con CVSS. Sin clave, 5 peticiones/30 s (con clave gratuita, 50). El vigilante **revisa además los rangos de versión** de cada CVE para descartar falsos positivos.

Fuente	Para qué	Cómo se consulta	Notas
OSV.dev (Google)	Vulnerabilidades de **paquetes**: Packagist, npm, Maven, PyPI, RubyGems, Go…	`POST /v1/querybatch` con nombre + versión exacta	Agrega GitHub Security Advisories, FriendsOfPHP, PyPA, Go… Da las **versiones que lo corrigen**. Hasta 1000 paquetes por petición.
CISA KEV	Saber si un CVE **se está explotando de verdad**	JSON público, se descarga cada 12 h	Si está en KEV, se trata como crítico aunque su CVSS sea medio.
EPSS (FIRST)	Probabilidad de explotación en los próximos 30 días	`api.first.org/data/v1/epss?cve=…`	Ayuda a separar «grave en teoría» de «me lo van a explotar».
endoflife.date	Saber si la rama ya **no recibe parches** (PHP 8.1, nginx 1.25…)	`/api/php.json`	Una rama sin soporte es un aviso por sí misma.

Por qué no solo NVD. NVD cubre bien los productos con CPE, pero los paquetes de Composer o npm están mucho mejor descritos en OSV/GitHub Advisories (con versión corregida exacta). Además, NVD arrastra retrasos de análisis desde 2024: OSV suele tener antes los avisos de paquetes.

Por qué no Trivy/Grype en esta primera versión. Son excelentes para escanear imágenes enteras, incluidos los paquetes del sistema operativo, pero pesan cientos de MB de base de datos y dan mucho ruido. Para este caso de uso (qué versión de PHP/Java/dependencias tiene mi web) basta con las APIs anteriores. Se pueden integrar más adelante como «capa F» (ver §9).

4. Cómo se decide qué es crítico

Cada hallazgo lleva CVSS, si está en KEV y su EPSS. Es **crítico** si se cumple cualquiera de:

- está en **CISA KEV** (se está explotando);
- **CVSS ≥ 9,0**;
- **CVSS ≥ 7,0 y EPSS ≥ 50 %**.

Los umbrales se cambian en `[reglas]`. Se pueden **ignorar** IDs concretos (falsos positivos revisados, o casos en los que la función vulnerable no se usa).

5. Avisos («avisa siempre»)

- **Consola/logs del contenedor**: siempre.
- **Telegram**, **webhook** (genérico, Slack o Discord; vale para n8n) y **email**: opcionales.
- Para no saturar: un **mensaje urgente por cada crítica** (hasta 5 por ciclo, primero las que ya se explotan) y **un resumen agrupado por componente** con la versión mínima a la que hay que subir para corregir todo.
- **No se repite** lo ya avisado. Sí se vuelve a avisar si una vulnerabilidad **escala** (p. ej. entra en KEV), y hay un **recordatorio cada 24 h** de las críticas sin resolver.
- Cuando una vulnerabilidad desaparece (se ha actualizado), llega un aviso de «ya no aplica».

6. Corrección con Claude: proponer, verificar, aprobar y aplicar

Reglas fundamentales:

1. **Claude solo trabaja cuando el usuario lo pide.** Tras cada análisis el vigilante únicamente avisa y ofrece la resolución (botón en Telegram y botón «Resolver con Claude» por componente o por app en el panel). No se llama a la API hasta que el usuario pulsa y confirma, en una ventana que explica qué se envía a Anthropic (vulnerabilidades y ficheros de despliegue; nunca los `.env` ni los datos) y que consume uso de la API. El modo antiguo, que preparaba planes solo, queda como opción (`[claude] modo = "auto"`) y no es el valor por defecto.
2. **NINGÚN cambio se aplica a la web sin una aprobación explícita del usuario.**

Claude solo propone. El vigilante verifica la propuesta en una copia, sin tocar la web. El usuario aprueba o rechaza cada cambio, o todos en bloque, desde Telegram o el panel. Solo entonces se aplica, siempre con una copia de seguridad previa y con posibilidad de volver atrás.

6.1 Flujo completo

```
nuevas vulnerabilidades en un componente (p. ej. php 8.1.2: 40 CVE)
       │  el vigilante AVISA y OFRECE; Claude no se usa todavía
       ▼
 EL USUARIO PIDE la resolución: «Resolver con Claude» (panel, o Telegram con
   doble confirmación) y confirma qué se enviará a Anthropic
       │
       ▼
① Plan de corrección (uno por componente)        estado: analizando
② Revisión automática de compatibilidad (registros de paquetes, endoflife)
③ Claude entiende y propone cambios + justificación + compatibilidad
       │
       ▼
④ Verificación en una COPIA del proyecto          estado: verificando
   copia → git apply → composer/npm en contenedor → pruebas de humo
   ¿falla? → Claude corrige el plan con el error (1 vez) → se verifica otra vez
       │
       ▼
⑤ Se comunica al usuario (Telegram + panel)       estado: pendiente_aprobacion
   un mensaje por cambio: qué arregla · por qué es correcto · riesgo ·
   compatibilidad · resultado de la verificación · cómo revertir
       │
       ├── rechazar (por cambio o todo) ──────► rechazado (no se toca nada)
       ▼
⑥ aprobar (por cambio o todo) + confirmar «Aplicar»
       │   si se aprobó solo una parte → se vuelve a verificar esa parte
       ▼
⑦ Aplicar                                          estado: aplicando
   copia de seguridad (ficheros + etiqueta de la imagen en marcha)
   → comprobar que producción no cambió desde la verificación
   → git apply del diff VERIFICADO → desplegar → comprobar la web (HTTP)
       │
       ├── la web responde mal ──► rollback automático ──► fallo_aplicacion
       ▼
    aplicado ──► re-escaneo (las vulnerabilidades deben desaparecer)
       │
       └── «Revertir» (panel o Telegram, con confirmación) ──► revertido
```

6.2 Qué recibe Claude y qué devuelve

Entrada:

- el componente con su versión exacta y dónde se detectó;
- las demás versiones desplegadas (PHP, Node…), que importan para la compatibilidad;
- todas las vulnerabilidades del componente, con CVSS, KEV, EPSS y versión corregida;
- la **revisión automática de compatibilidad** (§6.3);
- qué podrá comprobar el verificador;
- los ficheros de despliegue (`composer.json`, `package.json`, `pom.xml`, `Dockerfile`, `docker-compose.yml`, configuración de nginx o PHP; de los `.lock` solo el extracto del paquete) y el código fuente, para que busque usos de funciones deprecadas.

Claude puede además **consultar en la web** los changelogs, `UPGRADE.md` y las guías de migración oficiales (herramientas `web_search` y `web_fetch` del servidor de Anthropic). Se le pide usar fuentes primarias y citarlas.

Salida: un JSON validado por la API mediante un esquema (structured outputs), así que el vigilante no tiene que interpretar texto libre:

Campo	Contenido
`entendimiento`	resumen de las vulnerabilidades, versiones afectadas, versión corregida, si esta web está expuesta, CVE cubiertos
`version_objetivo`, `estrategia`	a qué versión ir y por qué
`cambios[]`	cada cambio, **con su justificación propia**:
· `tipo`	actualizar dependencia · actualizar imagen · configuración · código · mitigación temporal
· `diff`, `comandos_herramienta`	diff para `git apply` y, si hay que regenerar los lock, `composer update…` / `npm install…`
· `que_arregla`, `por_que_es_correcto`	la justificación
· `riesgo_nivel` (bajo/medio/alto), `riesgo_detalle`	qué puede romperse
· `breaking_changes`, `dependencias`, `deprecaciones`	la revisión de compatibilidad de ese cambio
· `como_revertir`, `depende_de`	marcha atrás y orden entre cambios
`pruebas_recomendadas`, `fuentes_consultadas`	qué conviene probar a mano y de dónde ha sacado la información

Modelo `claude-opus-5-5` con pensamiento adaptativo, esfuerzo alto y streaming. Si el modelo rechaza la petición, la API la reintenta sola con el modelo de respaldo. Las descripciones de los CVE, los ficheros y las páginas web se le marcan como **datos, no instrucciones** (protección contra inyección de instrucciones). Aun así, nada de lo que diga Claude se ejecuta fuera de la lista blanca de §6.4.

6.3 Revisión de compatibilidad: automática y por Claude

Antes de llamar a Claude, el vigilante saca **datos objetivos** de los registros. Así Claude no tiene que adivinar, y el usuario ve las alertas aunque Claude no las mencione:

- **Tipo de salto**: parche, menor o **mayor**. Un salto mayor casi seguro trae cambios incompatibles.
- **Opciones de versión**: la mínima segura, la última de la misma versión mayor y la última estable.
- **Packagist**: el `require.php` de la versión de destino frente al PHP desplegado, si el paquete está abandonado y qué dependencias nuevas añade.
- **npm**: `engines.node` frente al Node desplegado, si la versión está marcada como `deprecated` y sus `peerDependencies`.
- **Runtimes**: cambio de rama (PHP 8.1 → 8.2), con enlace a la guía oficial de migración, y si la rama de destino tiene soporte.
- **Cobertura**: qué CVE **no** corrige la versión que propone Claude. Ejemplo en la prueba: «8.2.29 no corrige CVE-2026-109618, corregido en 8.2.35».

Claude completa esta revisión con lo que no sale de los registros: cambios incompatibles documentados en los changelogs, funciones deprecadas que **este** código usa y riesgos de configuración.

6.4 Verificación en una copia y en contenedores de prueba

Todo pasa en `/datos/pruebas/<plan>/`, **sin tocar la web**:

1. **Copia** del proyecto (sin `.git`, `vendor`, `node_modules` ni `.env`) y un repositorio git temporal en la copia.
2. **Cada cambio por orden**: `git apply` de su diff. Si un cambio anterior tocó el mismo fichero, se aplica sobre el original y se fusiona con `git merge-file`. Si tiene comandos de herramienta, se ejecutan en un **contenedor desechable** (`composer:2`, `node:22-alpine`…) con la carpeta del proyecto, y se traen de vuelta los `.lock` regenerados. Después se hace commit: cada cambio queda con su **diff final**, lock incluido, que es exactamente lo que se aplicará.
3. **Comprobaciones estáticas**: JSON válidos, imágenes sin `latest` y que ningún diff se salga del proyecto.
4. **Pruebas de humo** por cada servicio configurado (`[[verificacion.servicios]]`):
    - se construye su imagen con los cambios, o se descarga la nueva imagen del `docker-compose.yml`;
    - se arranca con su configuración y se espera al healthcheck;
    - se ejecutan los comandos de prueba (`php -l`, `nginx -t`…) y peticiones HTTP básicas;
    - se buscan en los logs errores fatales (fallo) y deprecaciones (aviso).
5. Si algo falla, se repiten las pruebas con la copia **sin cambios**. Así se sabe si el fallo lo provoca el cambio o ya existía antes.
6. Si falla, **Claude recibe el resultado** (pasos y salidas) y corrige el plan **una vez**. El nuevo plan se verifica de nuevo y el panel muestra los dos intentos.

Seguridad de la verificación:

- Los comandos que propone Claude solo se ejecutan si pasan una **lista blanca**: `composer`, `npm`, `yarn`, `bundle`, `go` o `pip` con argumentos simples, sin shell, sin `;`, `|`, `$()`.
- Se ejecutan siempre con los scripts desactivados (`--no-scripts --no-plugins`, `--ignore-scripts`) y en un contenedor que se borra al terminar.
- Los comandos de las pruebas de humo los define **el usuario** en la configuración, nunca Claude.

Resultado de la verificación:

Estado	Significado
superada	todo correcto
parcial	algún cambio no se pudo preparar, el resto pasa
fallida	algo falla
solo estática	no hay Docker de pruebas: solo copia y comprobaciones estáticas, y **los cambios que necesitan regenerar un lock no se pueden aplicar**

6.5 Comunicación y aprobación

Telegram (bot con botones):

- **Cabecera del plan**: qué ha entendido Claude, si la web está expuesta, la versión de destino, el resultado de cada paso de la verificación y las deprecaciones vistas. Botones **«Aprobar todo y aplicar»** y **«Rechazar todo»**.
- **Un mensaje por cambio**: qué arregla, por qué es correcto, riesgo, cambios incompatibles, dependencias, deprecaciones, si se aplica limpio en la copia y cómo revertir. Botones **«Aprobar»** y **«Rechazar»**.
- Cuando ya no quedan cambios por decidir: **«Aplicar los N aprobados»**.
- Todo lo que toca la web (aplicar, aprobar todo y aplicar, revertir) pide una **segunda confirmación**.
- **Solo se atiende** a quien escribe desde el chat configurado y, si se configura, a los usuarios autorizados. El resto recibe «No autorizado» y no se registra nada.
- Los mensajes se actualizan con la decisión y el estado (APLICADO, REVERTIDO…), y los botones desaparecen cuando ya no tienen sentido.

Panel (`/plan/<id>`): lo mismo, más los diffs (el propuesto y el final verificado), las salidas de cada paso, las verificaciones anteriores y el **registro de auditoría**: quién decidió qué y cuándo. Las acciones piden el token del panel, que queda en una cookie `HttpOnly; SameSite=Strict` durante 12 horas, y una casilla de confirmación.

Webhook y email reciben un resumen con el enlace al panel. Las decisiones solo se toman en el panel o en Telegram.

6.6 Aplicación, copia de seguridad y marcha atrás

Solo se aplica tras pulsar «Aplicar» o «Aprobar todo y aplicar». Pasos:

1. **Comprobaciones previas**:
    - el plan sigue pendiente y hay cambios aprobados;

    - sus dependencias (`depende_de`) también están aprobadas;
    - lo que se va a aplicar es **exactamente lo verificado**: si se aprobó un subconjunto distinto, se verifica ese subconjunto antes;
    - los ficheros de producción **no han cambiado** desde la verificación (comparando hashes).
2. **Copia de seguridad** en `/datos/respaldos/<plan>-<fecha>/`: un `.tar.gz` con los ficheros afectados y la lista de ficheros nuevos. Además, la **imagen que está corriendo** se etiqueta como `vigilante-respaldo:<plan>-<contenedor>`, para volver exactamente a ella si hace falta.
3. `git apply` del diff final verificado sobre el proyecto. Opcionalmente, commit en git.
4. **Despliegue**: el comando configurado (en el ejemplo, `docker compose up -d --build --no-deps php nginx`) o un webhook a tu CI/CD. Si no hay ninguno, se avisa de que falta desplegar.
5. **Comprobación de la web** por HTTP. En el ejemplo se usa un puerto interno de nginx que no pasa por el modo mantenimiento, para poder comprobar aunque esté activo.
6. Si la web no responde bien y `rollback_automatico = true`: se **restauran los ficheros**, se **vuelve a desplegar** la versión anterior y se comprueba de nuevo. Es lo único que el vigilante hace sin pedir permiso, y solo para dejar las cosas como estaban.
7. Se avisa del resultado. Si se aplicó, se re-escanea para confirmar que las vulnerabilidades han desaparecido.

Revertir a mano («Revertir» en Telegram o en el panel, con confirmación) hace los pasos 6 en cualquier momento posterior.

6.7 Permisos: lo mínimo para cada parte

Parte	Necesita	En el ejemplo
Detección	listar e inspeccionar contenedores	`docker-proxy` de solo lectura
Verificación	construir y arrancar contenedores de prueba	`docker-proxy-pruebas` o, **mejor**, un Docker de otro equipo de pruebas (`VIGILANTE_DOCKER_PRUEBAS`)
Aplicación	escribir en el proyecto y desplegar	proyecto montado en lectura y escritura + `docker compose` a través de `docker-proxy-pruebas`, o un webhook a CI/CD

El proxy con escritura equivale casi a root sobre el servidor. La opción más segura es:

- verificar en un equipo de pruebas aparte;
- desplegar mediante el CI/CD (`webhook_despliegue`);
- montar el código en solo lectura.

En ese caso el vigilante solo deja preparado y verificado el diff, y la aplicación la hace el pipeline tras la aprobación.

7. Modo mantenimiento

Mecanismo: el vigilante escribe un fichero bandera `ACTIVO` en un volumen compartido con el servidor web. nginx lo comprueba en cada petición y, si existe, responde **503** con una página de mantenimiento que también genera el vigilante. No se para ni se toca ningún contenedor.

```
error_page 503 @mantenimiento;
location @mantenimiento { root /mantenimiento; rewrite ^ /mantenimiento.html break; }
location / {
    if (-f /mantenimiento/ACTIVO) { return 503; }
    ...
}
```

Hay **tres modos** (`[mantenimiento].modo`):

- `off`: nunca.
- `manual` (**por defecto**): ante una crítica, el aviso trae un enlace al panel con el botón «Poner la web en mantenimiento» (protegido con token).
- `auto`: se activa sola ante una crítica. Por defecto exige además que esté en **KEV o con EPSS alto**, para no tirar la web por un CVSS 9,8 teórico.

Alternativas para otras arquitecturas: ejecutar un **comando** o hacer **POST a un webhook** (balanceador, Traefik, Cloudflare…).

8. Prototipo: qué hay y qué se ha probado

Código (`vigilante/`): Python 3.12 con la librería estándar y `git`. El SDK de Anthropic es la única dependencia y solo hace falta para las propuestas. Unas 4.000 líneas:

Módulo	Función
`detectar_docker.py`	API de Docker por socket Unix o proxy TCP: inspect, exec, build, contenedores de prueba, archivos y etiquetas
`detectar_ficheros.py`	composer.lock, package-lock, yarn.lock, pom.xml, requirements, Gemfile.lock, go.mod, Dockerfile, WordPress
`detectar_http.py`	cabeceras y meta generator
`catalogo.py`	14 runtimes: variables de entorno, imágenes, CPE de NVD, slug de endoflife
`fuentes.py`	NVD, OSV, KEV, EPSS, endoflife.date (con caché en disco y respeto de límites)
`motor.py`	ciclo, deduplicado, reglas, estado, avisos, mantenimiento automático
`compatibilidad.py`	revisión automática: saltos de versión, Packagist, npm, ramas, cobertura de CVE
`claude.py`	análisis con salida estructurada, investigación web y corrección tras un fallo
`planes.py`	planes, estados, decisiones por cambio o en bloque, auditoría
`verificacion.py`	copia, git apply y fusión, herramientas en contenedor, lista blanca, pruebas de humo
`aplicar.py`	comprobaciones previas, copia de seguridad, aplicación, despliegue, comprobación, rollback
`telegram.py`	bot con botones, solo chat y usuarios autorizados, confirmaciones
`correcciones.py`	

Módulo	Función
	orquesta el flujo; panel y Telegram usan las mismas funciones
`panel.py`	panel web: estado, planes con diffs y justificación, aprobar, rechazar, aplicar y revertir

Panel web (sin dependencias externas, modo claro y oscuro, adaptado a móvil, sin emoticonos; los estados se muestran con iconos SVG y texto, nunca solo con color):

- **Desplegable de apps** (cuando hay más de una): cada proyecto de docker compose es una app. El selector filtra todas las páginas (resumen, vulnerabilidades, componentes y correcciones), recuerda la elección y muestra, por app, vulnerabilidades, críticas y correcciones pendientes. En «todas» el resumen incluye una tarjeta por app. Cada plan de corrección es de una app, con su carpeta de proyecto y su comprobación de salud; sin código montado de una app no se prepara plan. El modo mantenimiento es global del servidor.
- **Resumen**: cifras clave, gráfico de vulnerabilidades por componente y gravedad (con detalle al pasar el ratón), estado de la web, ramas sin soporte, correcciones pendientes y lo más urgente.
- **Vulnerabilidades**: tabla con búsqueda, filtros por gravedad y «solo explotadas», EPSS como barra y paginación.
- **Componentes**: versión, origen de la detección, soporte de la rama y distribución de gravedad.
- **Correcciones** y **detalle de cada plan**:
  - etapas del plan (análisis, verificación, tu decisión, aplicación) y barra de decisión fija;
  - cada cambio con su justificación, su compatibilidad, el diff coloreado y sus botones de aprobar y rechazar;
  - la verificación paso a paso, el registro de auditoría y diálogos de confirmación antes de aplicar, revertir o rechazar todo.
- **Mantenimiento**: activar y desactivar con confirmación.
- Página pública de mantenimiento rediseñada.

Pruebas realizadas (con APIs reales, Docker simulado porque el equipo de pruebas no tiene Docker):

- Web de ejemplo con nginx 1.25.3 + PHP 8.1.2 + Guzzle 7.4.0, Twig 3.3.7, axios 0.21.0, lodash 4.17.15: **9 componentes, 118 vulnerabilidades, 14 críticas, 3 en KEV** (p. ej. CVE-2024-4577 en PHP-CGI). Detectó además que **PHP 8.1 y nginx 1.25 están sin soporte**. El ciclo tarda unos 40 s la primera vez (por el límite de NVD sin clave) y 1 s con caché.
- Proyecto Java con log4j-core 2.14.1 + Tomcat 9.0.50: el primer aviso urgente es **Log4Shell (CVE-2021-44228)**.
- Segundo ciclo sin cambios: **0 avisos repetidos**.
- Mantenimiento: activación manual desde el panel (token incorrecto → 403) y automática.
- **Flujo de corrección completo** (`pruebas/prueba_flujo.py`, **49/49 comprobaciones**). Usa Docker, la API de Claude (en streaming, como la real) y Telegram simulados; NVD, OSV y los registros de paquetes son reales. Comprueba:
  - se crean planes para PHP y Twig y **el proyecto no cambia** hasta aprobar;
  - la petición a Claude lleva salida estructurada, herramientas web, fallback y la revisión de compatibilidad;

    - verificación con imagen construida, comandos de prueba, HTTP, nginx sacado del compose y `composer update` en contenedor con el lock regenerado;
    - se detecta que 8.2.29 no corrige dos CVE;
    - un chat no autorizado es ignorado;
    - aprobar el cambio 1 y rechazar el 2: se re-verifica solo el 1 y se aplica solo el 1, con copia de seguridad, etiqueta de imagen y despliegue;
    - revertir desde el panel: sin confirmación o con token malo no hace nada;
    - Twig con la web rota tras desplegar: **rollback automático** y la web vuelve a responder;
    - un plan que falla en la copia: Claude recibe el error, lo corrige y pasa.
- 15 pruebas unitarias sin red (`python3 -m unittest discover -s pruebas`): restricciones de versión, lista blanca de comandos, partición de diffs y decisiones.
- **Falta**:
    - probarlo con una clave real de Anthropic: aquí no hay ninguna, y el formato de la petición se ha comprobado contra la API simulada;
    - probarlo con un Docker y un bot de Telegram reales.

9. Limitaciones conocidas y siguientes pasos

1. **Backports de distribuciones**: si nginx viene de un paquete de Debian/Ubuntu (no de la imagen oficial), la versión «1.18.0» puede tener parches aplicados y NVD dará falsos positivos. Solución: capa F con los paquetes del SO (`dpkg`/`apk`) contra OSV (ecosistemas Debian/Alpine/Ubuntu) o integrar Trivy.
2. **Paquetes del sistema** (openssl, glibc, libxml2…): no se miran todavía (misma capa F).
3. **Java empaquetado** (`.war`/`.jar` sin `pom.xml`): leer `META-INF/maven/**/pom.properties`.
4. **Pull request** como forma de aplicar: en vez de escribir en el servidor, abrir una PR con el diff verificado y su justificación. La aprobación sería el merge, y el CI/CD desplegaría.
5. **Tests del propio proyecto** en la verificación (`composer test`, `npm test`), además de las pruebas de humo, si el proyecto los tiene.
6. **Ventana de mantenimiento**: dejar aprobado un cambio para que se aplique a una hora concreta.
7. **Varias webs** desde un solo vigilante y página de estado agregada.
8. **VEX / justificación**: marcar «no afecta porque no usamos esa función» desde el panel en lugar de editar la lista `ignorar`.
9. Probar el `docker compose` completo en un servidor real con Docker.